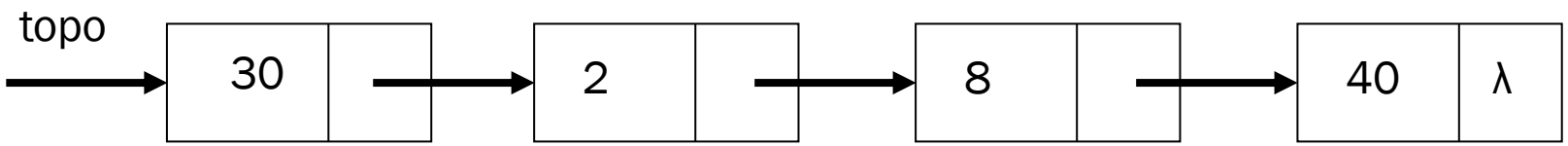
ESTRUTURAS DE DADOS BÁSICAS PARTE 3/3

PILHAS

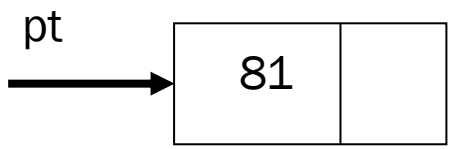
```
empilha(x)
        ocupar(pt)
         pt↑.chave ← x
         pt↑.prox ← topo
         topo ← pt
```

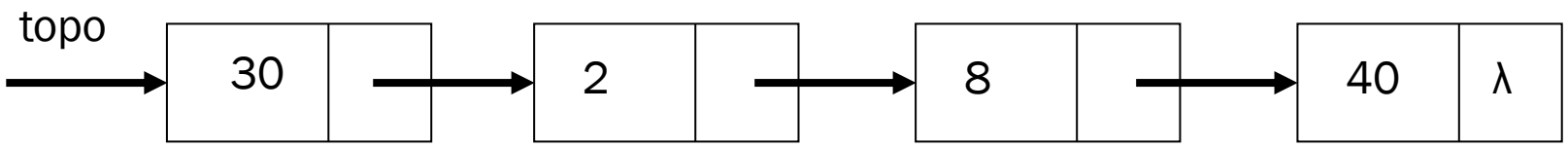


PILHAS

```
empilha(x)
        ocupar(pt)
         pt↑.chave ← x
         pt↑.prox ← topo
         topo ← pt
```

Exemplo: x=81

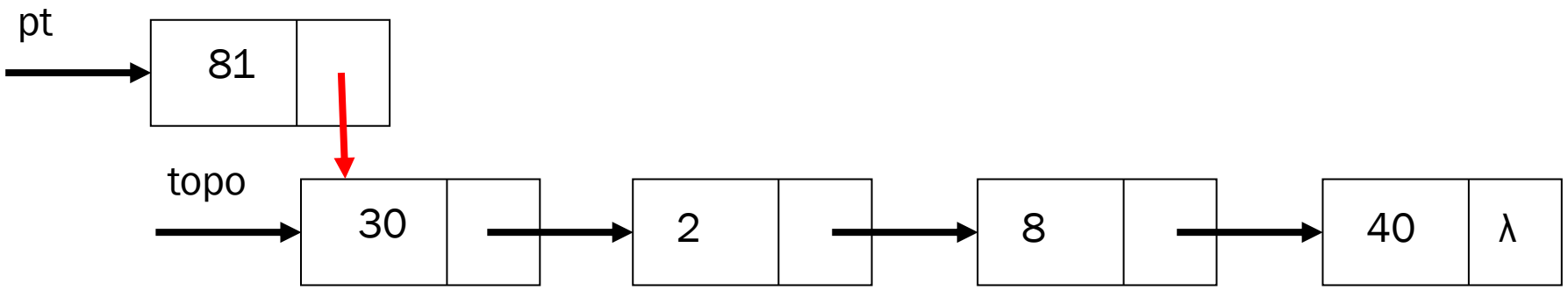




PILHAS

```
empilha(x)
        ocupar(pt)
         pt↑.chave ← x
         pt↑.prox ← topo
         topo ← pt
```

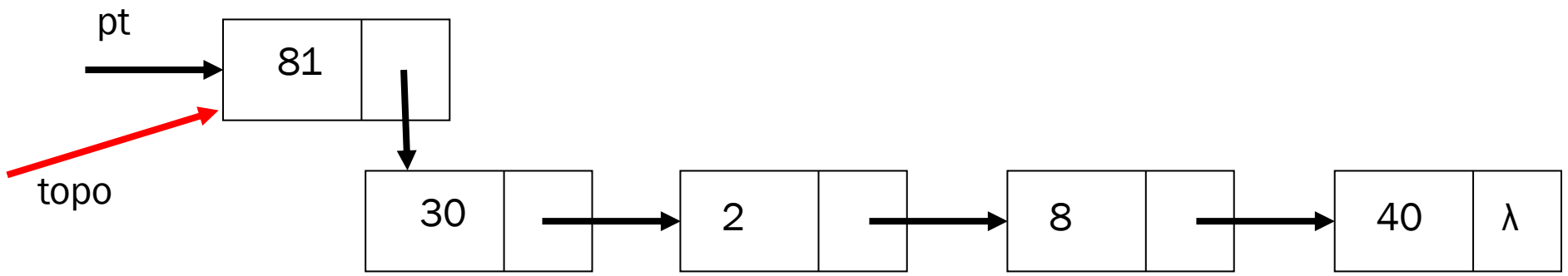
Exemplo: x=81



PILHAS

```
empilha(x)
        ocupar(pt)
         pt↑.chave ← x
         pt↑.prox ← topo
         topo ← pt
```

Exemplo: x=81



PILHAS

```
empilha(x)
        ocupar(pt)
         pt↑.chave ← x
         pt↑.prox ← topo
         topo ← pt
```

Complexidade



PILHAS

```
empilha(x)
        ocupar(pt)
         pt↑.chave ← x
         pt↑.prox ← topo
         topo ← pt
```

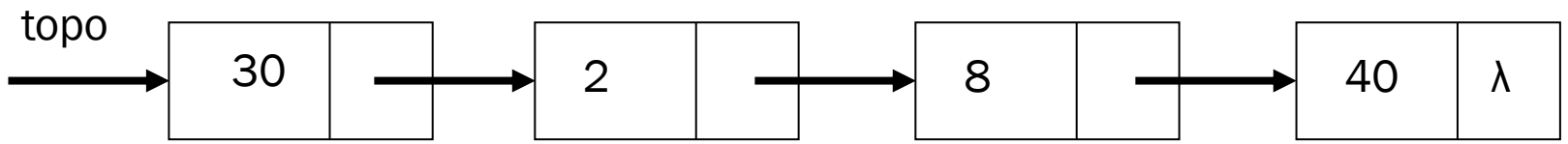
Todos os comandos: $\theta(1)$

Complexidade é $\theta(1)$



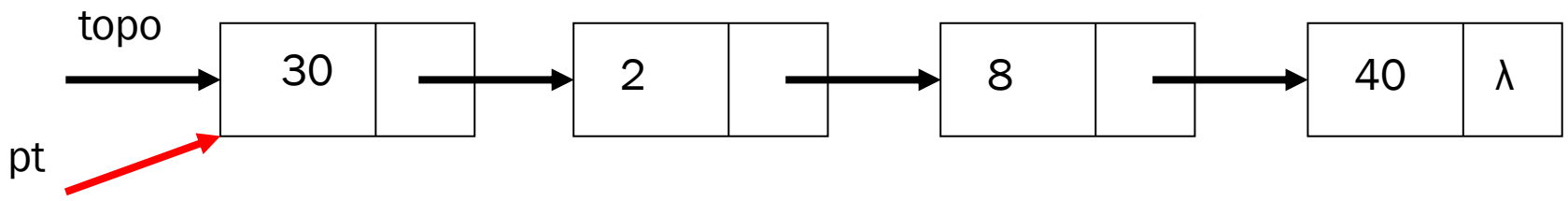
PILHAS

```
desempilha( )
    se (topo ≠ λ) então
        pt ← topo
        topo ← topo↑.prox
        valor ← pt↑.chave
       desocupar(pt)
     senão “Pilha vazia”
```



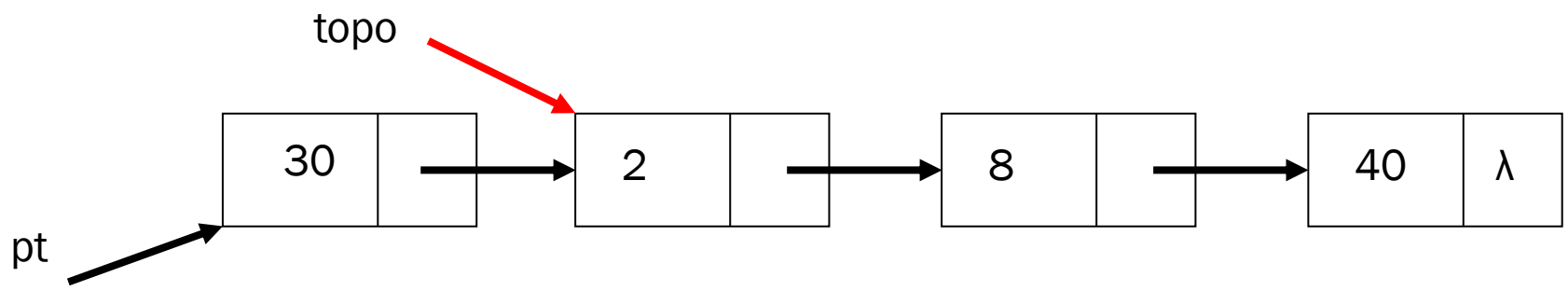
PILHAS

```
desempilha( )
    se (topo ≠ λ) então
        pt ← topo
        topo ← topo↑.prox
        valor ← pt↑.chave
       desocupar(pt)
     senão “Pilha vazia”
```



PILHAS

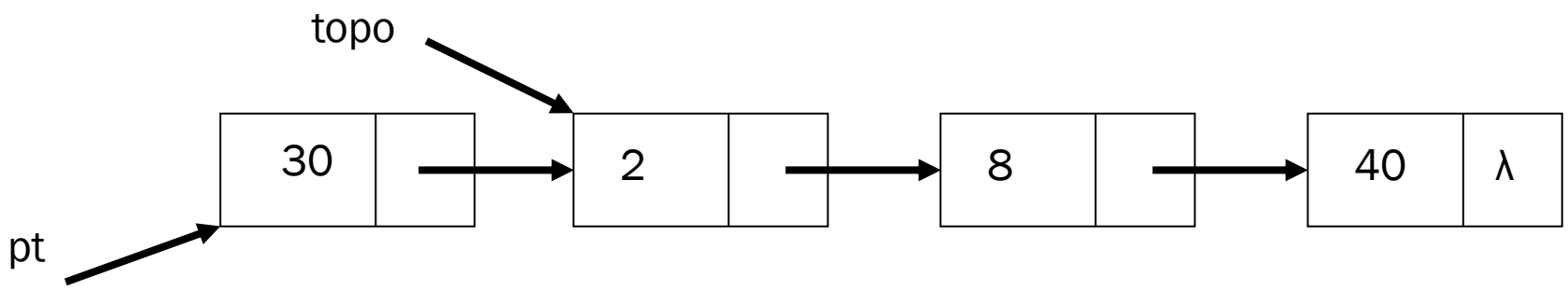
```
desempilha( )
    se (topo ≠ λ) então
        pt ← topo
        topo ← topo↑.prox
        valor ← pt↑.chave
        desocupar(pt)
    senão “Pilha vazia”
```



PILHAS

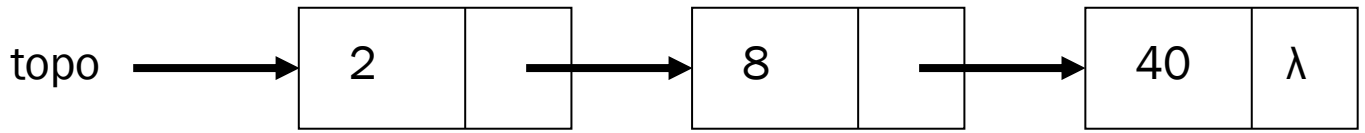
```
desempilha( )
    se (topo ≠ λ) então
        pt ← topo
        topo ← topo↑.prox
        valor ← pt↑.chave
       desocupar(pt)
     senão “Pilha vazia”
```

valor=30



PILHAS

```
desempilha( )
    se (topo ≠ λ) então
        pt ← topo
        topo ← topo↑.prox
        valor ← pt↑.chave
        desocupar(pt)
    senão “Pilha vazia”
```



PILHAS

```
desempilha( )
    se (topo ≠ λ) então
        pt ← topo
        topo ← topo↑.prox
        valor ← pt↑.chave
        desocupar(pt)
    senão “Pilha vazia”
```

Complexidade



PILHAS

```
desempilha( )
    se (topo ≠ λ) então
        pt ← topo
        topo ← topo↑.prox
        valor ← pt↑.chave
        desocupar(pt)
    senão “Pilha vazia”
```

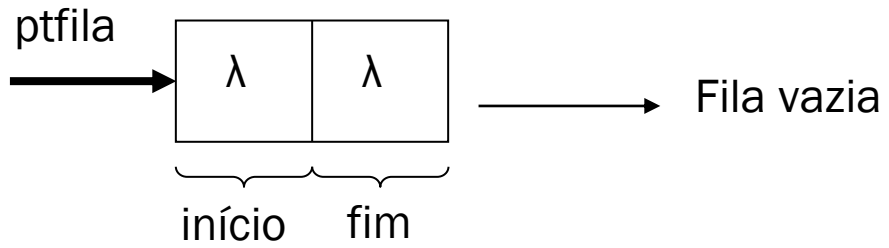
Todos os comandos: $\theta(1)$

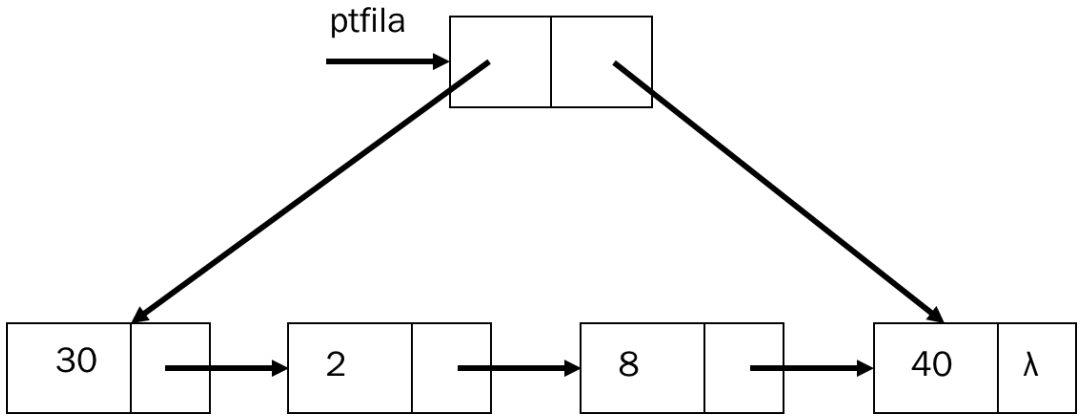
Complexidade é $\theta(1)$



FILAS

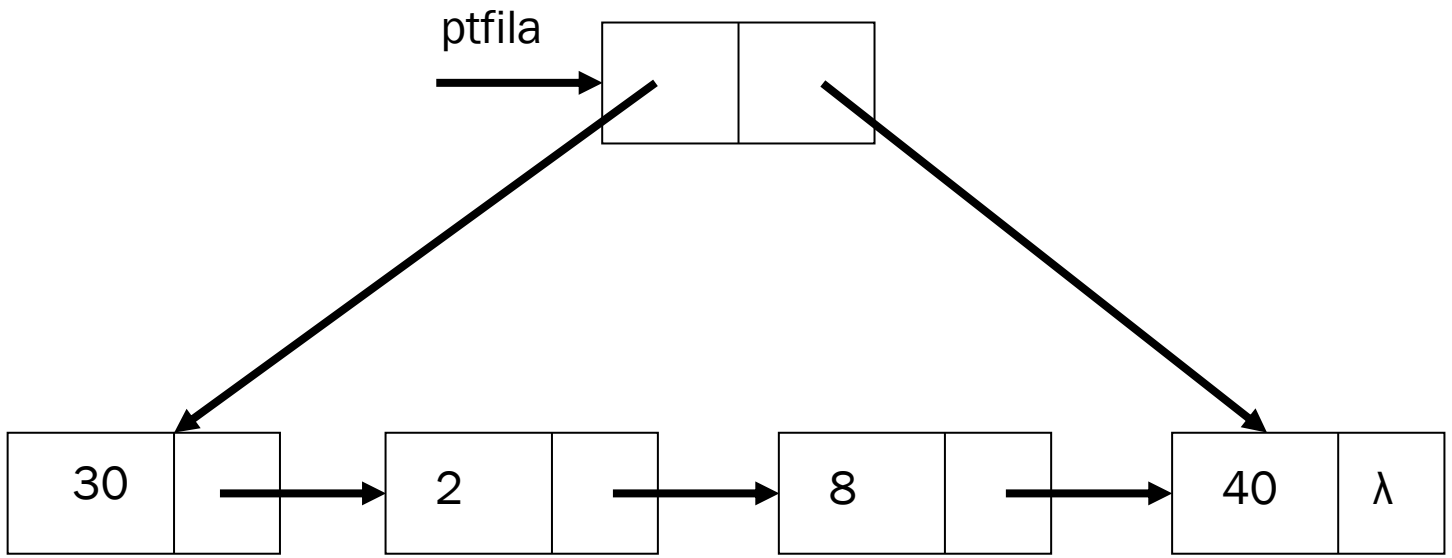
```
Insere(x)
        ocupar(pt)
         pt↑.chave ← x
         pt↑.prox ← λ
         se (ptfila↑.fim ≠ λ) então
              ptfila↑.fim↑.prox ← pt
         senão
              ptfila↑.inicio ← pt
         ptfila↑.fim ← pt
```

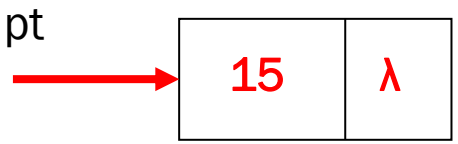




FILAS

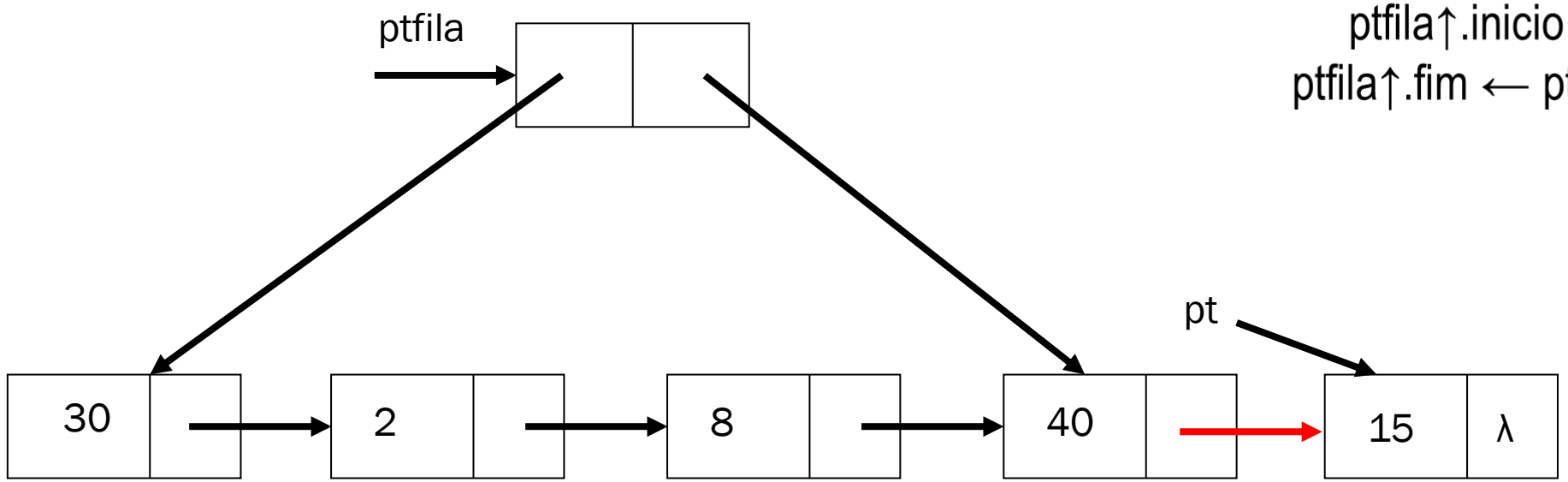
```
Insere(x)
        ocupar(pt)
         pt↑.chave ← x
         pt↑.prox ← λ
         se (ptfila↑.fim ≠ λ) então
             ptfila↑.fim↑.prox ← pt
         senão
             ptfila↑.inicio ← pt
         ptfila↑.fim ← pt
```





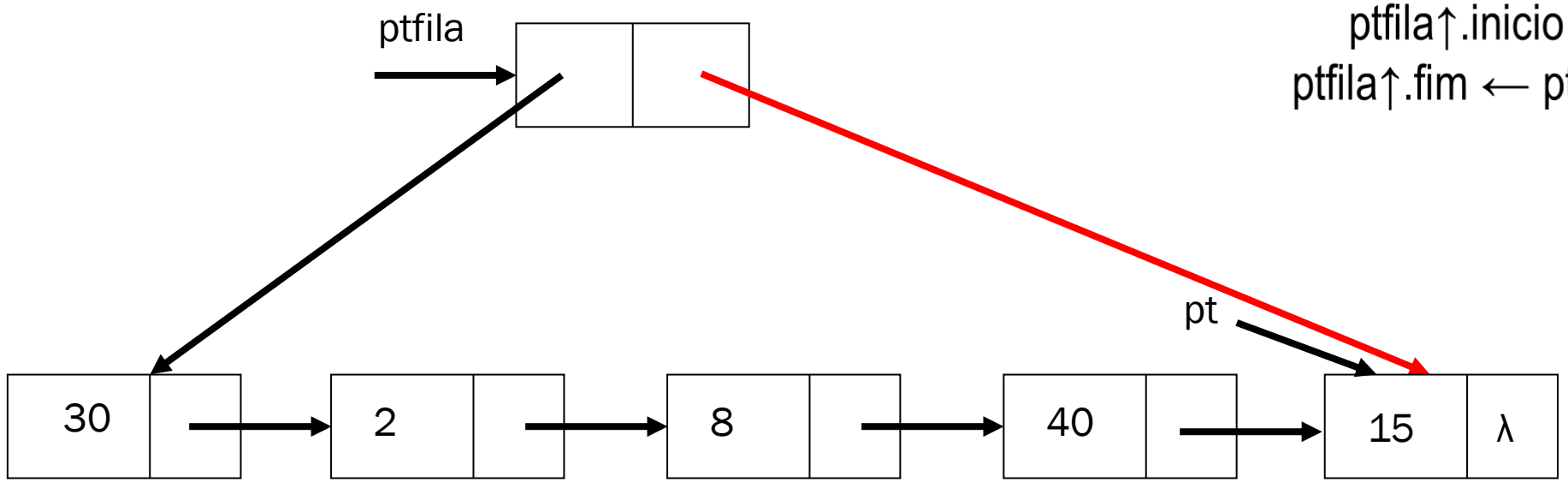
FILAS

```
Insere(x)
        ocupar(pt)
         pt↑.chave ← x
         pt↑.prox ← λ
         se (ptfila↑.fim ≠ λ) então
             ptfila↑.fim↑.prox ← pt
         senão
             ptfila↑.inicio ← pt
         ptfila↑.fim ← pt
```



FILAS

```
Insere(x)
        ocupar(pt)
         pt↑.chave ← x
         pt↑.prox ← λ
         se (ptfila↑.fim ≠ λ) então
             ptfila↑.fim↑.prox ← pt
         senão
             ptfila↑.inicio ← pt
         ptfila↑.fim ← pt
```



FILAS

```
Insere(x)
        ocupar(pt)
         pt↑.chave ← x
         pt↑.prox ← λ
         se (ptfila↑.fim ≠ λ) então
              ptfila↑.fim↑.prox ← pt
         senão
              ptfila↑.inicio ← pt
         ptfila↑.fim ← pt
```

Complexidade ?



FILAS

```
Insere(x)
        ocupar(pt)
         pt↑.chave ← x
         pt↑.prox ← λ
         se (ptfila↑.fim ≠ λ) então
              ptfila↑.fim↑.prox ← pt
         senão
              ptfila↑.inicio ← pt
         ptfila↑.fim ← pt
```

Todos os comandos: $\theta(1)$

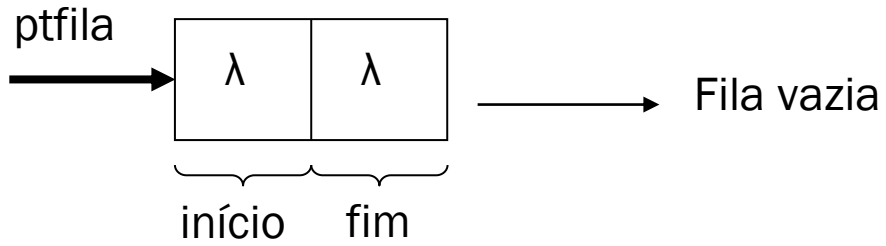
Complexidade é $\theta(1)$

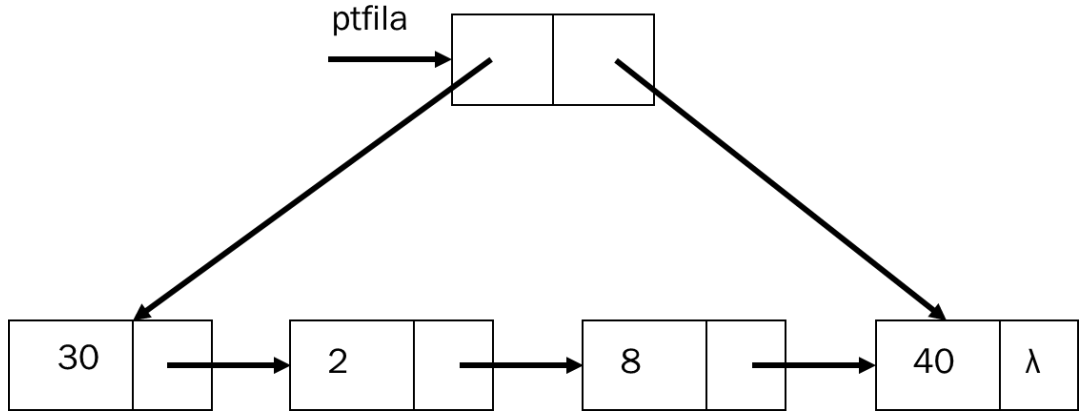


FILAS

Remove( )

```
se (ptfila↑.inicio ≠ λ) então
    pt ← ptfila↑.inicio
    ptfila↑.inicio ← ptfila↑.inicio↑.prox
    se (ptfila↑.inicio = λ) então
        ptfila↑.fim ← λ
    valor ← pt↑.chave
    desocupar(pt)
senão “Fila Vazia”
```

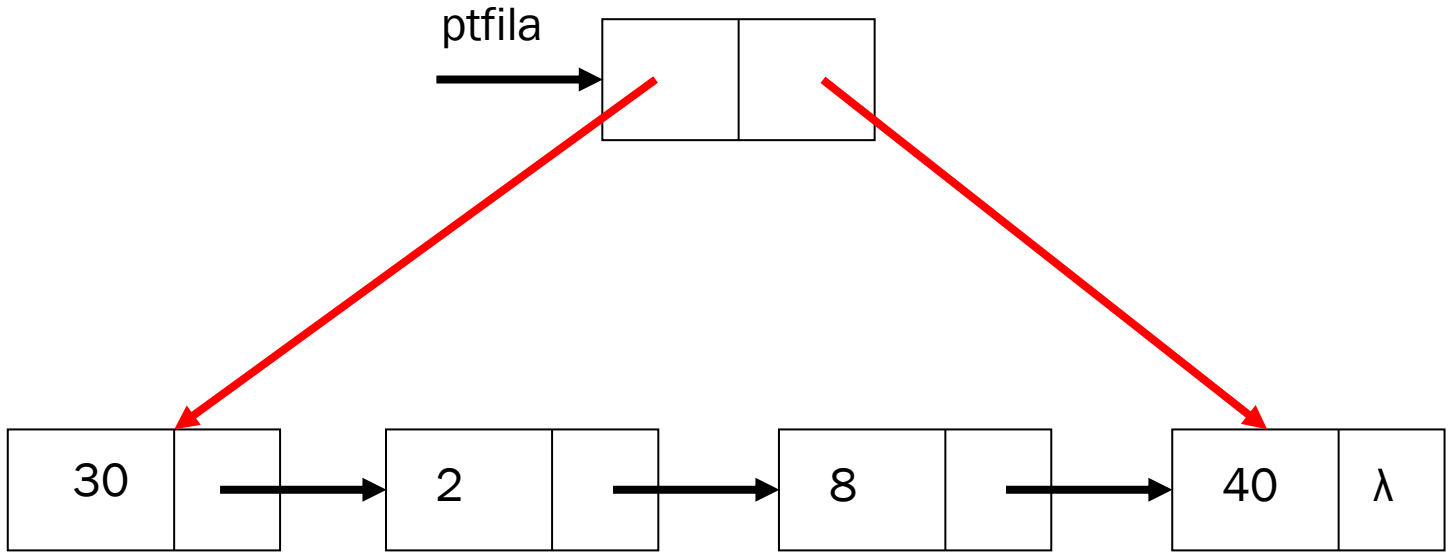




FILAS

Remove( )

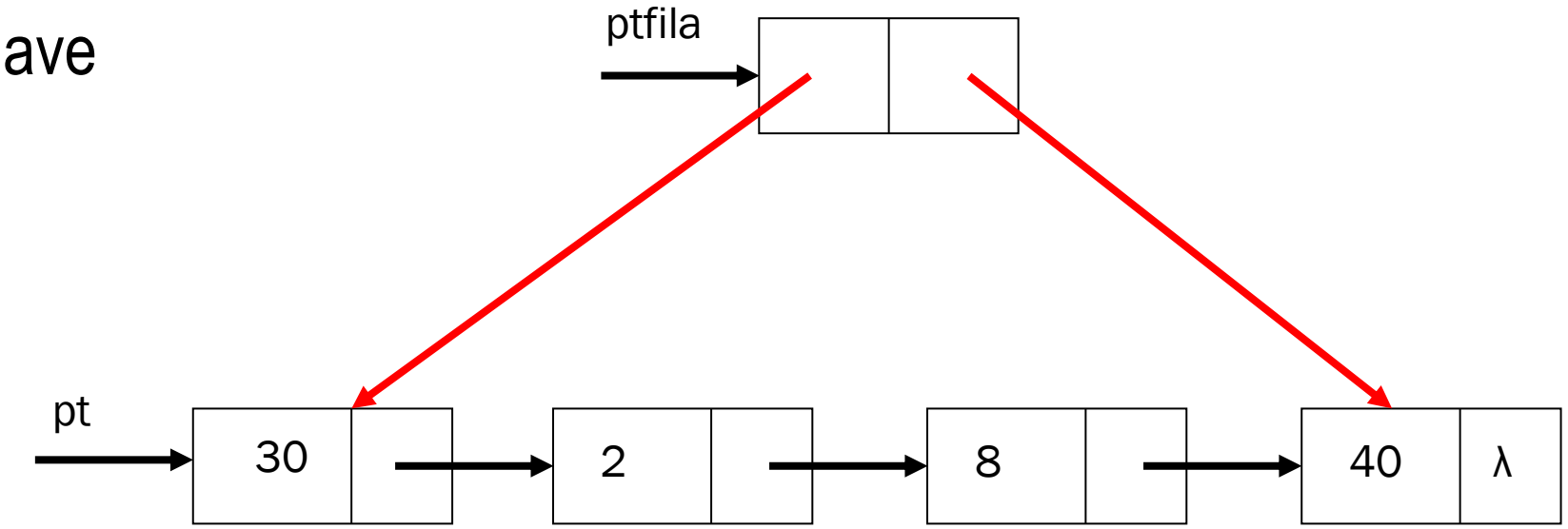
```
se (ptfila↑.inicio ≠ λ) então
    pt ← ptfila↑.inicio
    ptfila↑.inicio ← ptfila↑.inicio↑.prox
    se (ptfila↑.inicio = λ) então
        ptfila↑.fim ← λ
    valor ← pt↑.chave
    desocupar(pt)
senão “Fila Vazia”
```



FILAS

Remove( )

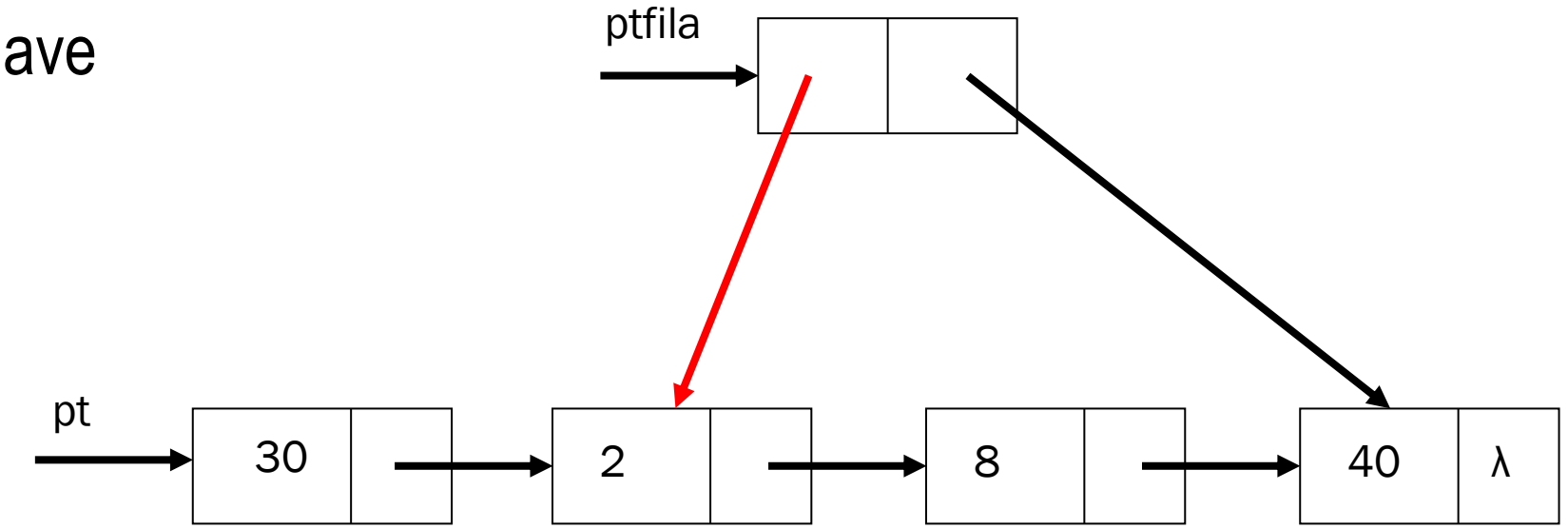
```
se (ptfila↑.inicio ≠ λ) então
    pt ← ptfila↑.inicio
    ptfila↑.inicio ← ptfila↑.inicio↑.prox
    se (ptfila↑.inicio = λ) então
        ptfila↑.fim ← λ
    valor ← pt↑.chave
    desocupar(pt)
senão “Fila Vazia”
```



FILAS

Remove( )

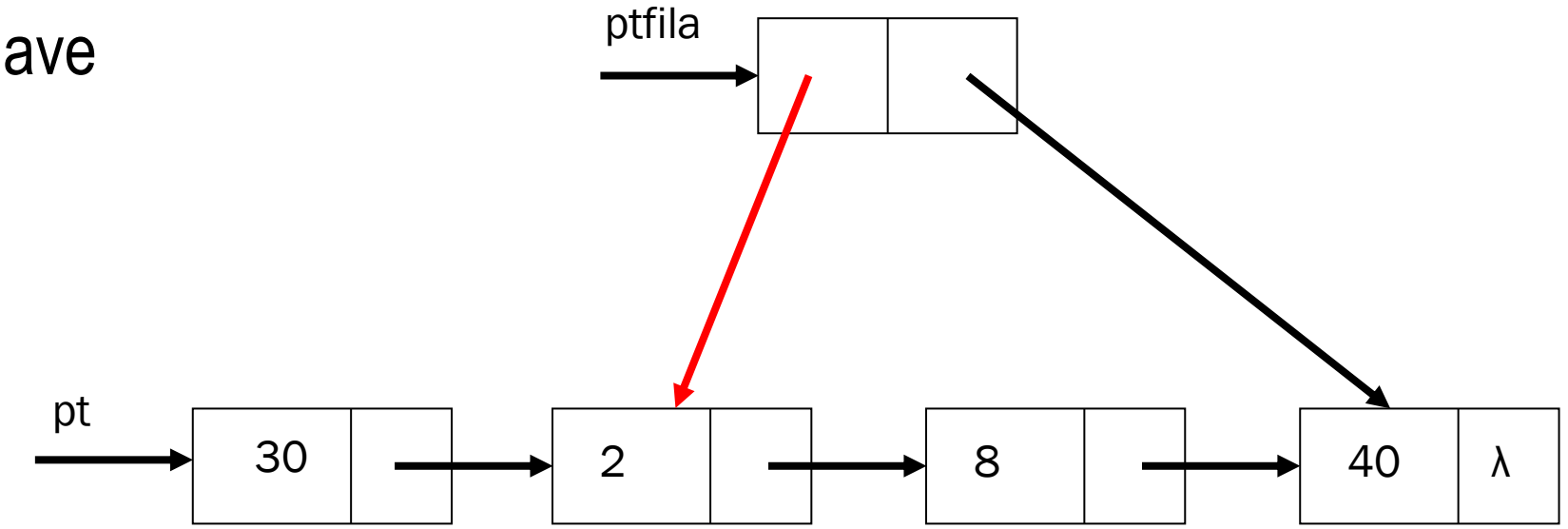
```
se (ptfila↑.inicio ≠ λ) então
    pt ← ptfila↑.inicio
    ptfila↑.inicio ← ptfila↑.inicio↑.prox
    se (ptfila↑.inicio = λ) então
        ptfila↑.fim ← λ
    valor ← pt↑.chave
    desocupar(pt)
senão “Fila Vazia”
```



FILAS

Remove( )

```
se (ptfila↑.inicio ≠ λ) então
    pt ← ptfila↑.inicio
    ptfila↑.inicio ← ptfila↑.inicio↑.prox
    se (ptfila↑.inicio = λ) então
        ptfila↑.fim ← λ
    valor ← pt↑.chave
    desocupar(pt)
senão “Fila Vazia”
```

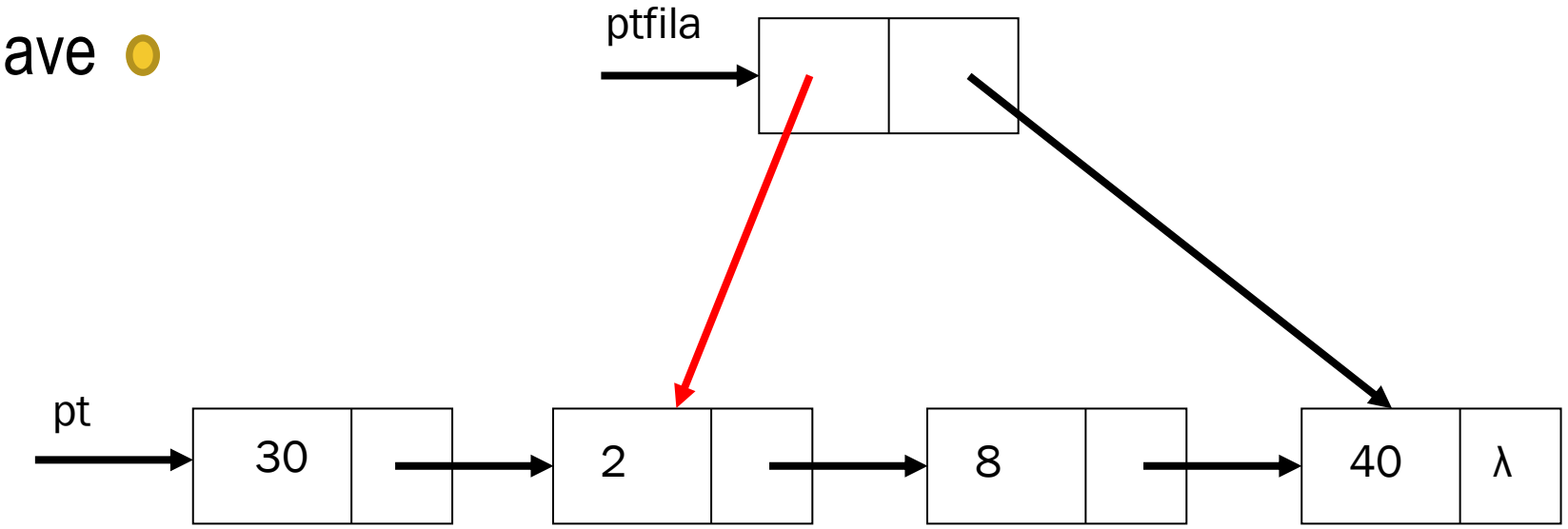


FILAS

Remove( )

```
se (ptfila↑.inicio ≠ λ) então
    pt ← ptfila↑.inicio
    ptfila↑.inicio ← ptfila↑.inicio↑.prox
    se (ptfila↑.inicio = λ) então
        ptfila↑.fim ← λ
    valor ← pt↑.chave
    desocupar(pt)
senão “Fila Vazia”
```

Valor = 30

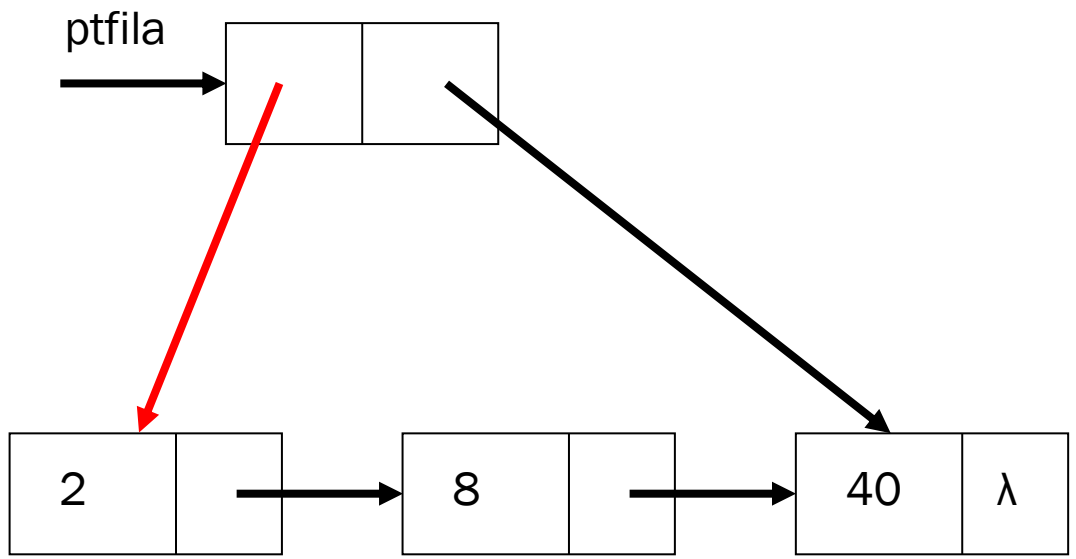


FILAS

Remove( )

```
se (ptfila↑.inicio ≠ λ) então
    pt ← ptfila↑.inicio
    ptfila↑.inicio ← ptfila↑.inicio↑.prox
    se (ptfila↑.inicio = λ) então
        ptfila↑.fim ← λ
    valor ← pt↑.chave
    desocupar(pt)
senão “Fila Vazia”
```

Valor = 30



FILAS

Remove( )

```
se (ptfila↑.inicio ≠ λ) então
    pt ← ptfila↑.inicio
    ptfila↑.inicio ← ptfila↑.inicio↑.prox
    se (ptfila↑.inicio = λ) então
        ptfila↑.fim ← λ
    valor ← pt↑.chave
    desocupar(pt)
senão “Fila Vazia”
```

Complexidade ?



FILAS

Remove( )

```
se (ptfila↑.inicio ≠ λ) então
    pt ← ptfila↑.inicio
    ptfila↑.inicio ← ptfila↑.inicio↑.prox
    se (ptfila↑.inicio = λ) então
        ptfila↑.fim ← λ
    valor ← pt↑.chave
    desocupar(pt)
senão “Fila Vazia”
```

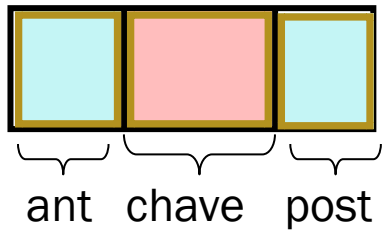
Todos os comandos: $\theta(1)$

Complexidade é $\theta(1)$



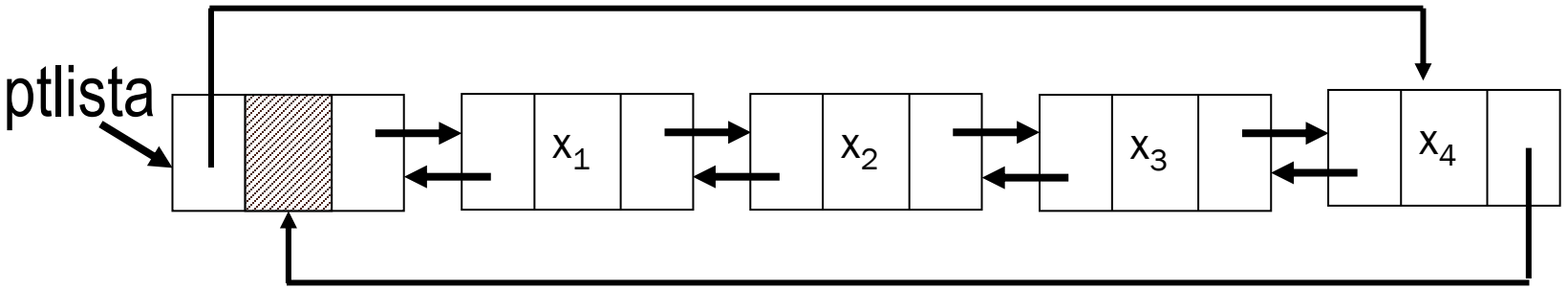
Lista Linear Duplamente Encadeada

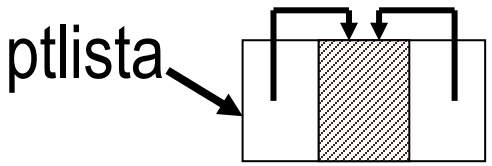
Tipo de variável apontada tem 3 campos:
chave (dado),
ant e post (ponteiros para variáveis do mesmo tipo)



Lista Linear Duplamente Encadeada

Exemplo (lista circular com nó cabeça)





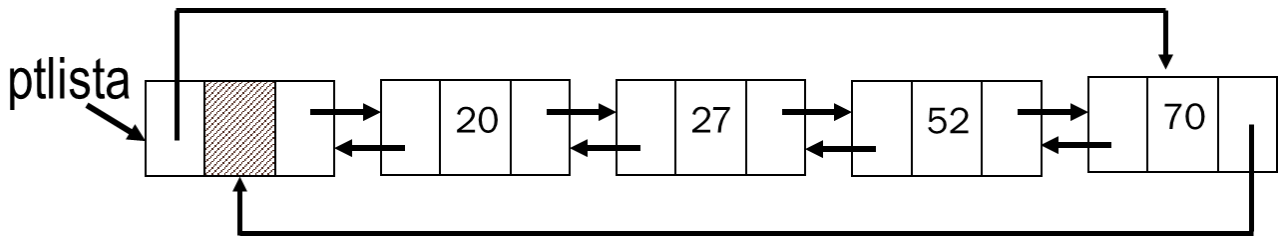
Lista Vazia

Lista Linear Duplamente Encadeada

Busca em Lista Ordenada

```
Busca(x) //lista duplamente encadeada com nó cabeça

    ultimo←ptlista↑.ant
    se (ultimo ≠ ptlista) e (ultimo↑.chave ≥ x) então
        pont ←ptlista↑.post
        enquanto(pont↑.chave < x) faça
                pont ←pont↑.post
        retorna(pont)
    senão retorna(ptlista)
```



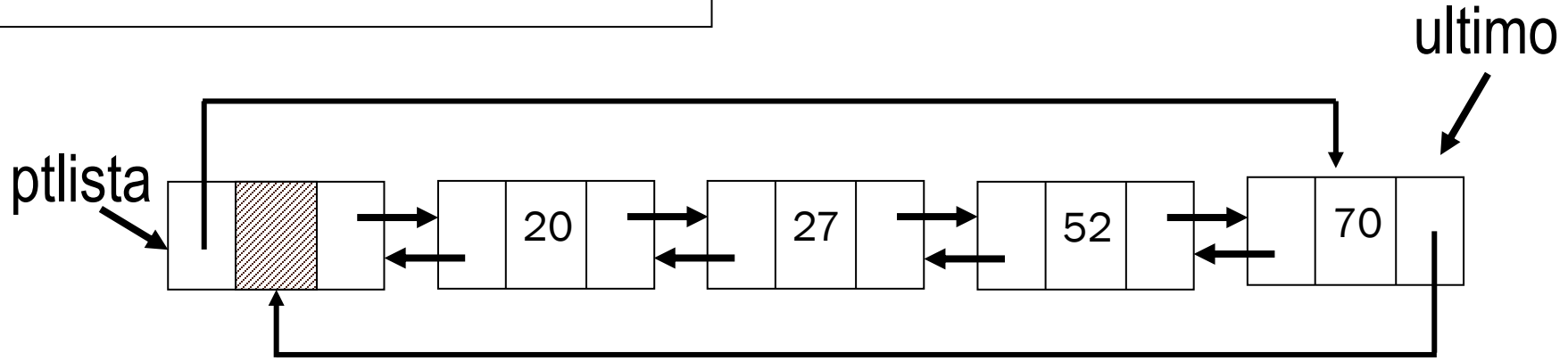
Lista Linear Duplamente Encadeada

Busca em Lista Ordenada

```
Busca(x) //lista duplamente encadeada com nó cabeça

    ultimo←ptlista↑.ant
    se (ultimo ≠ ptlista) e (ultimo↑.chave ≥ x) então
       pont ←ptlista↑.post
       enquanto(pont↑.chave < x) faça
             pont ←pont↑.post
       retorna(pont)
    senão retorna(ptlista)
```

x=27



Lista Linear Duplamente Encadeada

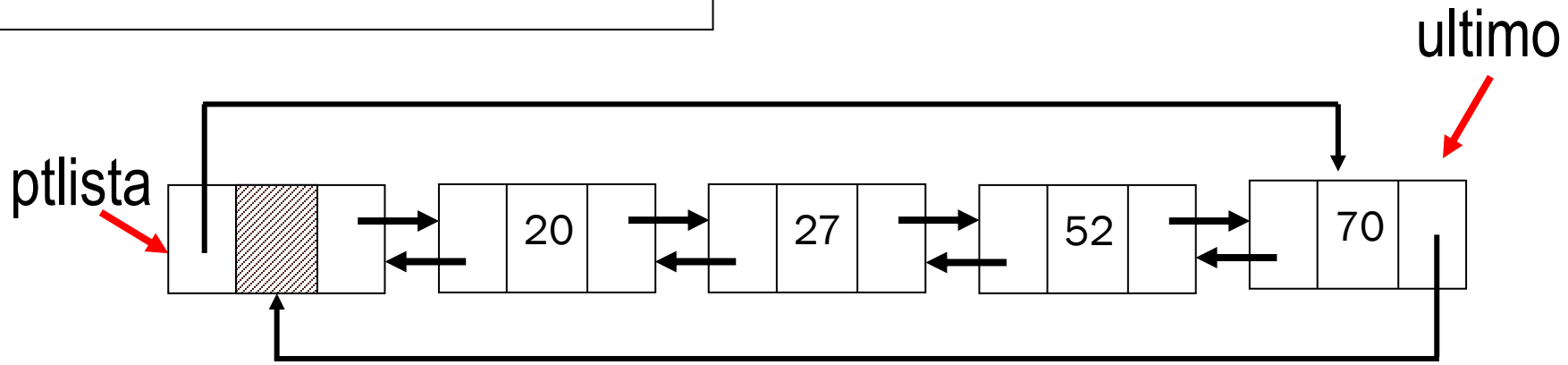
Busca em Lista Ordenada

```
Busca(x) //lista duplamente encadeada com nó cabeça

    ultimo←ptlista↑.ant
    se (ultimo ≠ ptlista) e (ultimo↑.chave ≥ x) então
        pont ←ptlista↑.post
        enquanto(pont↑.chave < x) faça
            pont ←pont↑.post
        retorna(pont)
    senão retorna(ptlista)
```

OK

x=27



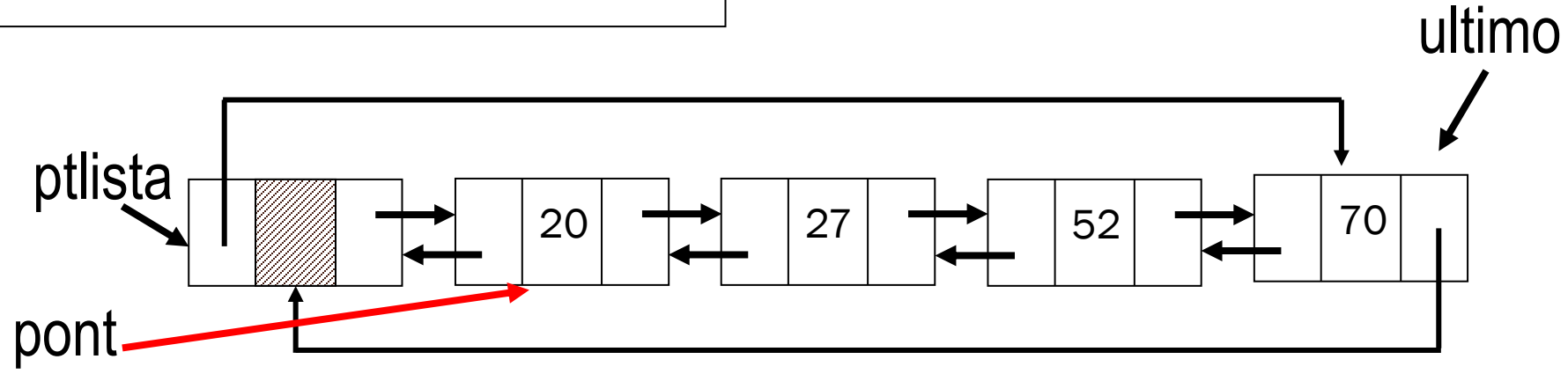
Lista Linear Duplamente Encadeada

Busca em Lista Ordenada

```
Busca(x) //lista duplamente encadeada com nó cabeça

    ultimo←ptlista↑.ant
    se (ultimo ≠ ptlista) e (ultimo↑.chave ≥ x) então
       pont ←ptlista↑.post
       enquanto(pont↑.chave < x) faça
            pont ←pont↑.post
       retorna(pont)
    senão retorna(ptlista)
```

x=27



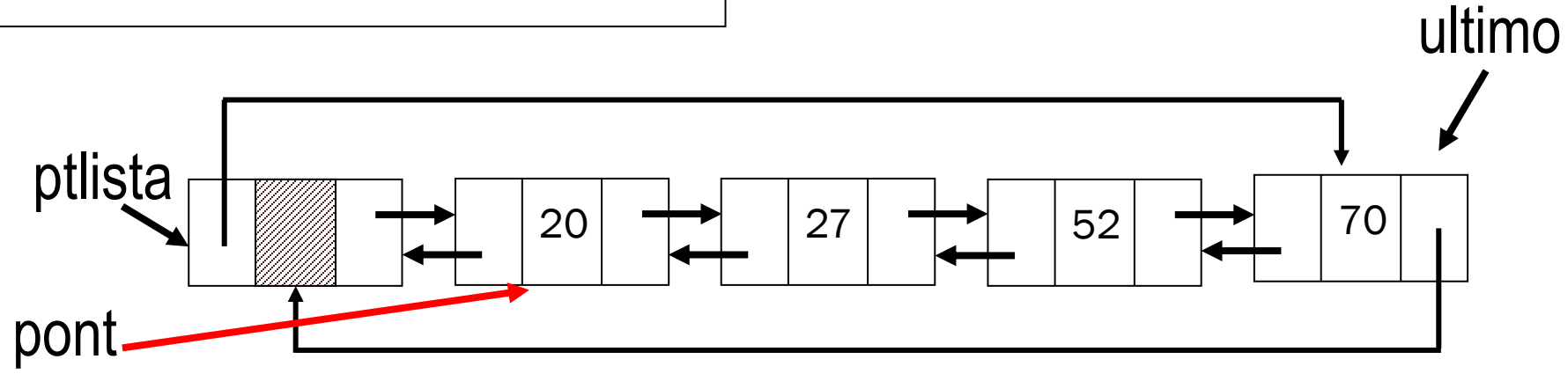
Lista Linear Duplamente Encadeada

Busca em Lista Ordenada

```
Busca(x) //lista duplamente encadeada com nó cabeça

    ultimo←ptlista↑.ant
    se (ultimo ≠ ptlista) e (ultimo↑.chave ≥ x) então
       pont ←ptlista↑.post
       enquanto(pont↑.chave < x) faça
             pont ←pont↑.post
       retorna(pont)
    senão retorna(ptlista)
```

x=27



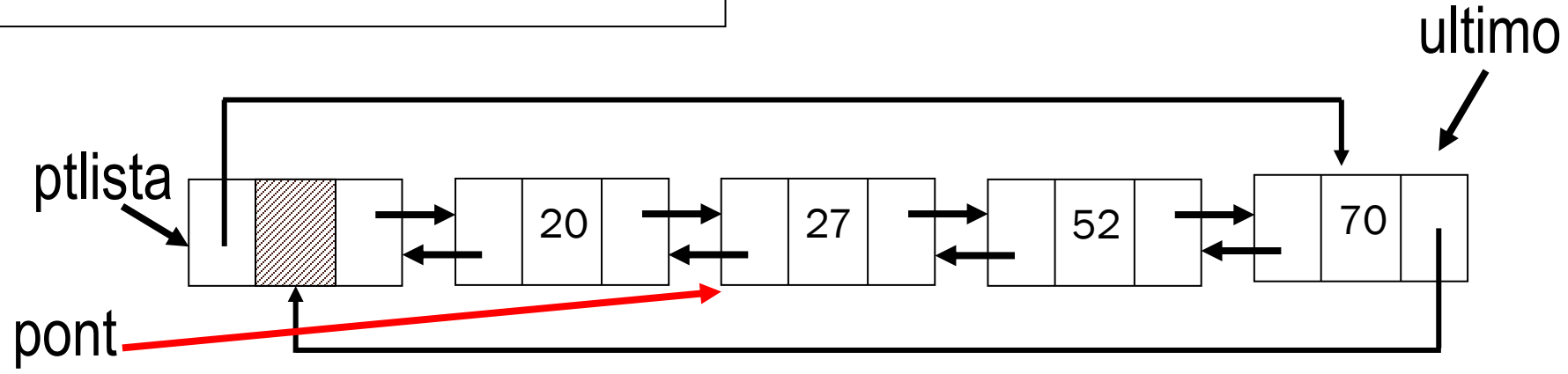
Lista Linear Duplamente Encadeada

Busca em Lista Ordenada

```
Busca(x) //lista duplamente encadeada com nó cabeça

    ultimo←ptlista↑.ant
    se (ultimo ≠ ptlista) e (ultimo↑.chave ≥ x) então
       pont ←ptlista↑.post
       enquanto(pont↑.chave < x) faça
            pont ←pont↑.post
       retorna(pont)
    senão retorna(ptlista)
```

x=27



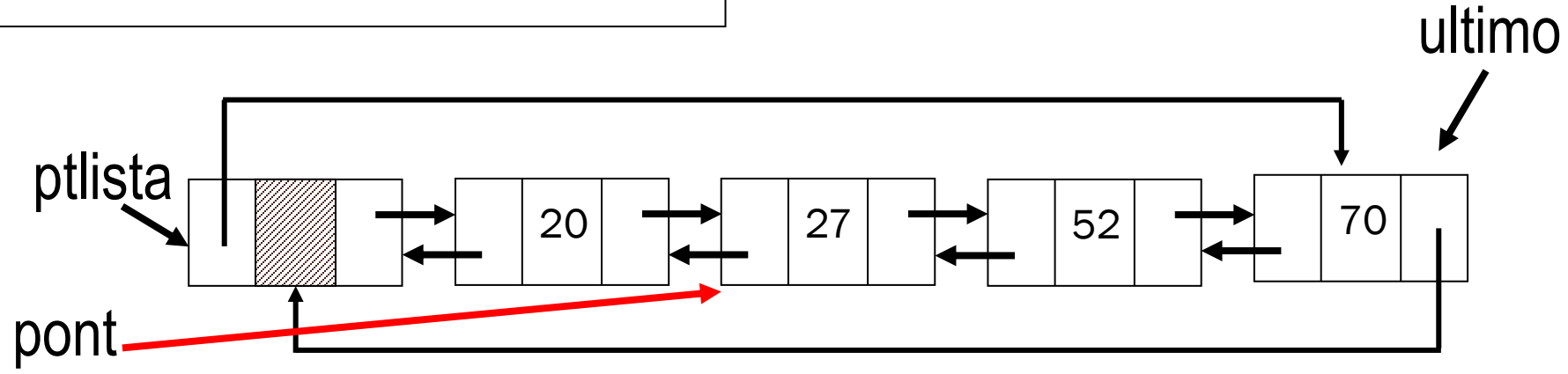
Lista Linear Duplamente Encadeada

Busca em Lista Ordenada

```
Busca(x) //lista duplamente encadeada com nó cabeça

    ultimo←ptlista↑.ant
    se (ultimo ≠ ptlista) e (ultimo↑.chave ≥ x) então
       pont ←ptlista↑.post
       enquanto(pont↑.chave < x) faça
            pont ←pont↑.post
       retorna(pont)
    senão retorna(ptlista)
```

x=27



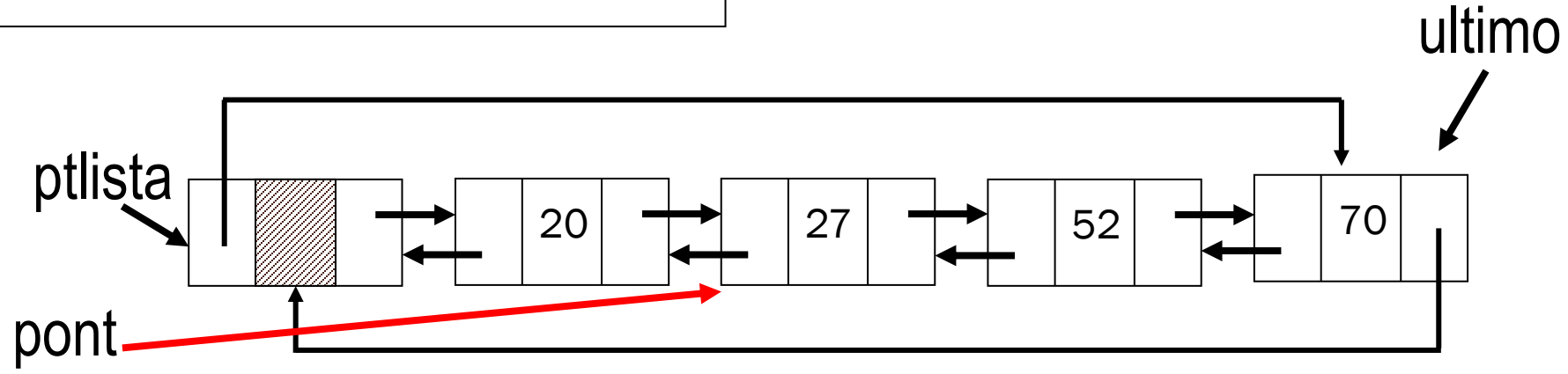
Lista Linear Duplamente Encadeada

Busca em Lista Ordenada

```
Busca(x) //lista duplamente encadeada com nó cabeça

    ultimo←ptlista↑.ant
    se (ultimo ≠ ptlista) e (ultimo↑.chave ≥ x) então
       pont ←ptlista↑.post
       enquanto(pont↑.chave < x) faça
            pont ←pont↑.post
       retorna(pont)
    senão retorna(ptlista)
```

x=27



Lista Linear Duplamente Encadeada

Busca em Lista Ordenada

```
Busca(x) //lista duplamente encadeada com nó cabeça

    ultimo←ptlista↑.ant
    se (ultimo ≠ ptlista) e (ultimo↑.chave ≥ x) então
        pont ←ptlista↑.post
        enquanto(pont↑.chave < x) faça
                pont ←pont↑.post
        retorna(pont)
    senão retorna(ptlista)
```

Complexidade ?



Lista Linear Duplamente Encadeada

Busca em Lista Ordenada

```
Busca(x) //lista duplamente encadeada com nó cabeça

    ultimo←ptlista↑.ant
    se (ultimo ≠ ptlista) e (ultimo↑.chave ≥ x) então
        pont ←ptlista↑.post
        enquanto(pont↑.chave < x) faça
                pont ←pont↑.post
        retorna(pont)
    senão retorna(ptlista)
```



Melhor caso

Encontra a chave de busca na primeira posição

ou

Chave de busca é maior que a chave do último elemento da lista

$\Omega(1)$

Lista Linear Duplamente Encadeada

Busca em Lista Ordenada

```
Busca(x) //lista duplamente encadeada com nó cabeça

    ultimo←ptlista↑.ant
    se (ultimo ≠ ptlista) e (ultimo↑.chave ≥ x) então
        pont ←ptlista↑.post
        enquanto(pont↑.chave < x) faça
                pont ←pont↑.post
        retorna(pont)
    senão retorna(ptlista)
```



Pior caso

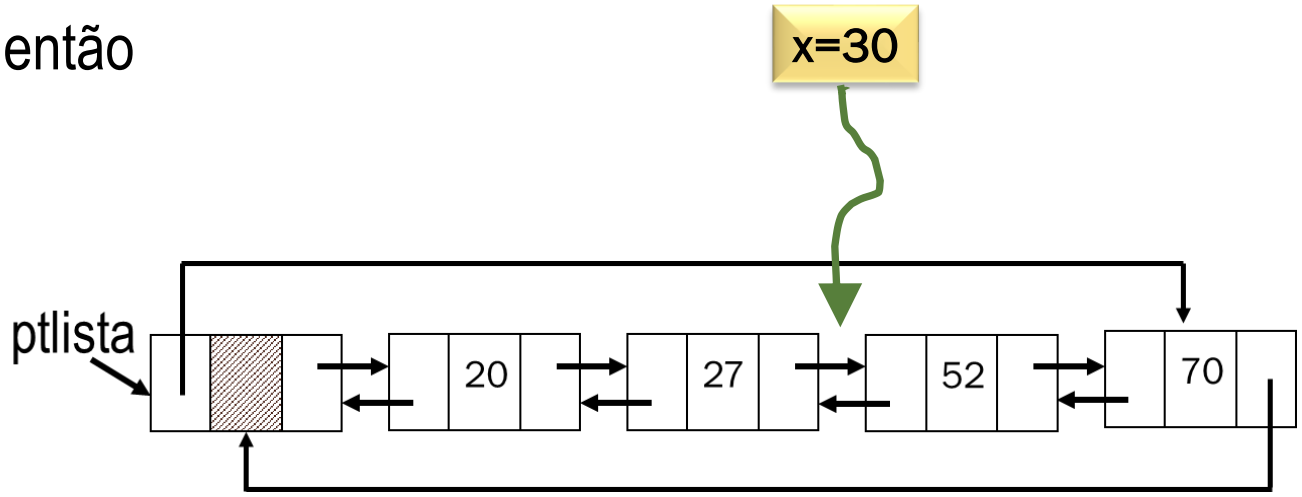
Encontra a chave de busca na última posição

$O(n)$

Lista Linear Duplamente Encadeada

Inserção

```
Insere(x)
    pont ← Busca(x)
    se (pont = ptlista) ou (pont↑.chave ≠ x) então
        anterior ← pont↑.ant
        ocupar(pt)
        pt↑.chave ← x
        pt↑.ant ← anterior
        pt↑.post ← pont
        anterior↑.post ← pt
        pont↑.ant ← pt
    senão “Elemento já se encontra na lista”
```

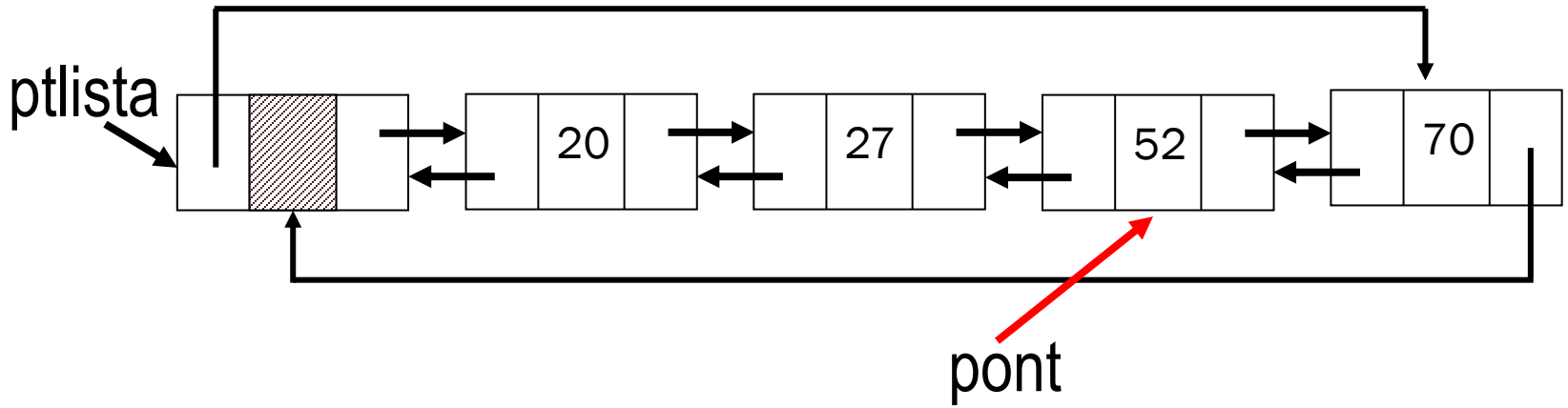


Lista Linear Duplamente Encadeada

Inserção

```
Insere(x)
    pont ← Busca(x)
    se (pont = ptlista) ou (pont↑.chave ≠ x) então
        anterior ← pont↑.ant
        ocupar(pt)
        pt↑.chave ← x
        pt↑.ant ← anterior
        pt↑.post ← pont
        anterior↑.post ← pt
        pont↑.ant ← pt
    senão "Elemento já se encontra na lista"
```

x=30



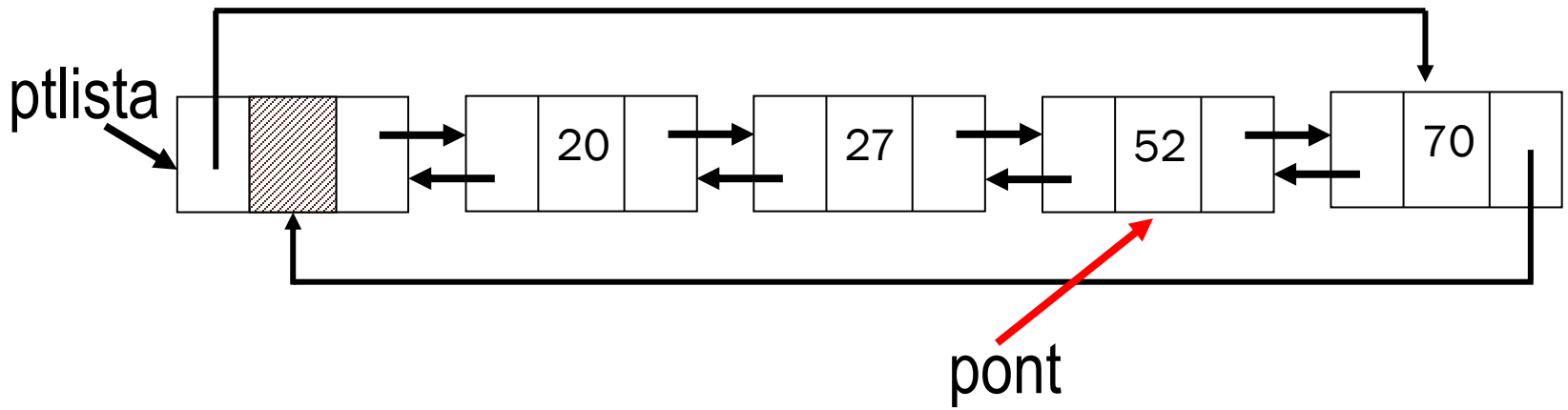
Lista Linear Duplamente Encadeada

Inserção

```
Insere(x)
    pont ← Busca(x)
    se (pont = ptlista) ou (pont↑.chave ≠ x) então
        anterior ← pont↑.ant
        ocupar(pt)
        pt↑.chave ← x
        pt↑.ant ← anterior
        pt↑.post ← pont
        anterior↑.post ← pt
        pont↑.ant ← pt
    senão "Elemento já se encontra na lista"
```

OK

x=30

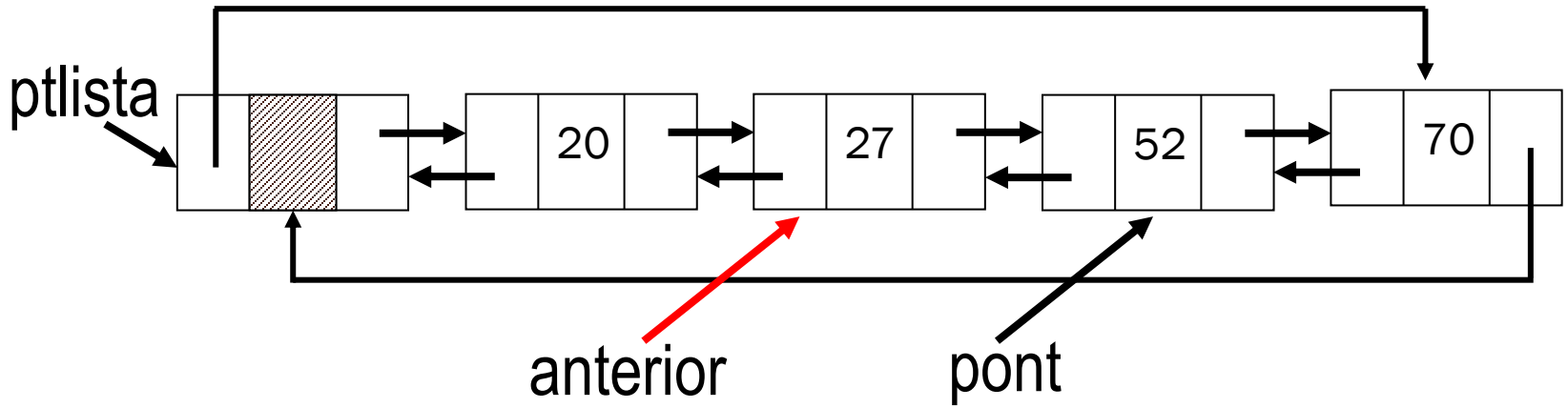


Lista Linear Duplamente Encadeada

Inserção

```
Insere(x)
    pont ← Busca(x)
    se (pont = ptlista) ou (pont↑.chave ≠ x) então
        anterior ← pont↑.ant
        ocupar(pt)
        pt↑.chave ← x
        pt↑.ant ← anterior
        pt↑.post ← pont
        anterior↑.post ← pt
        pont↑.ant ← pt
    senão "Elemento já se encontra na lista"
```

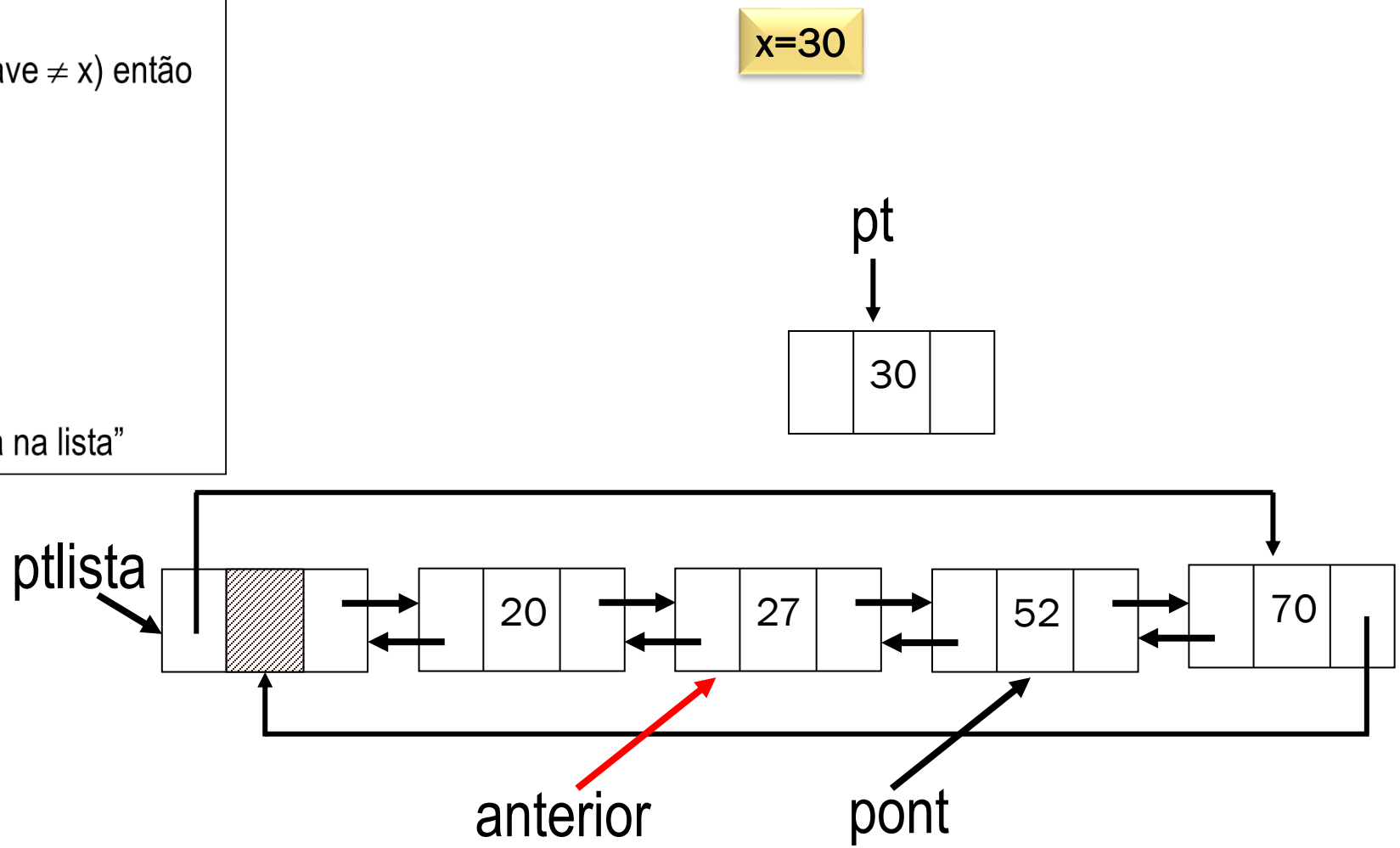
x=30



Lista Linear Duplamente Encadeada

Inserção

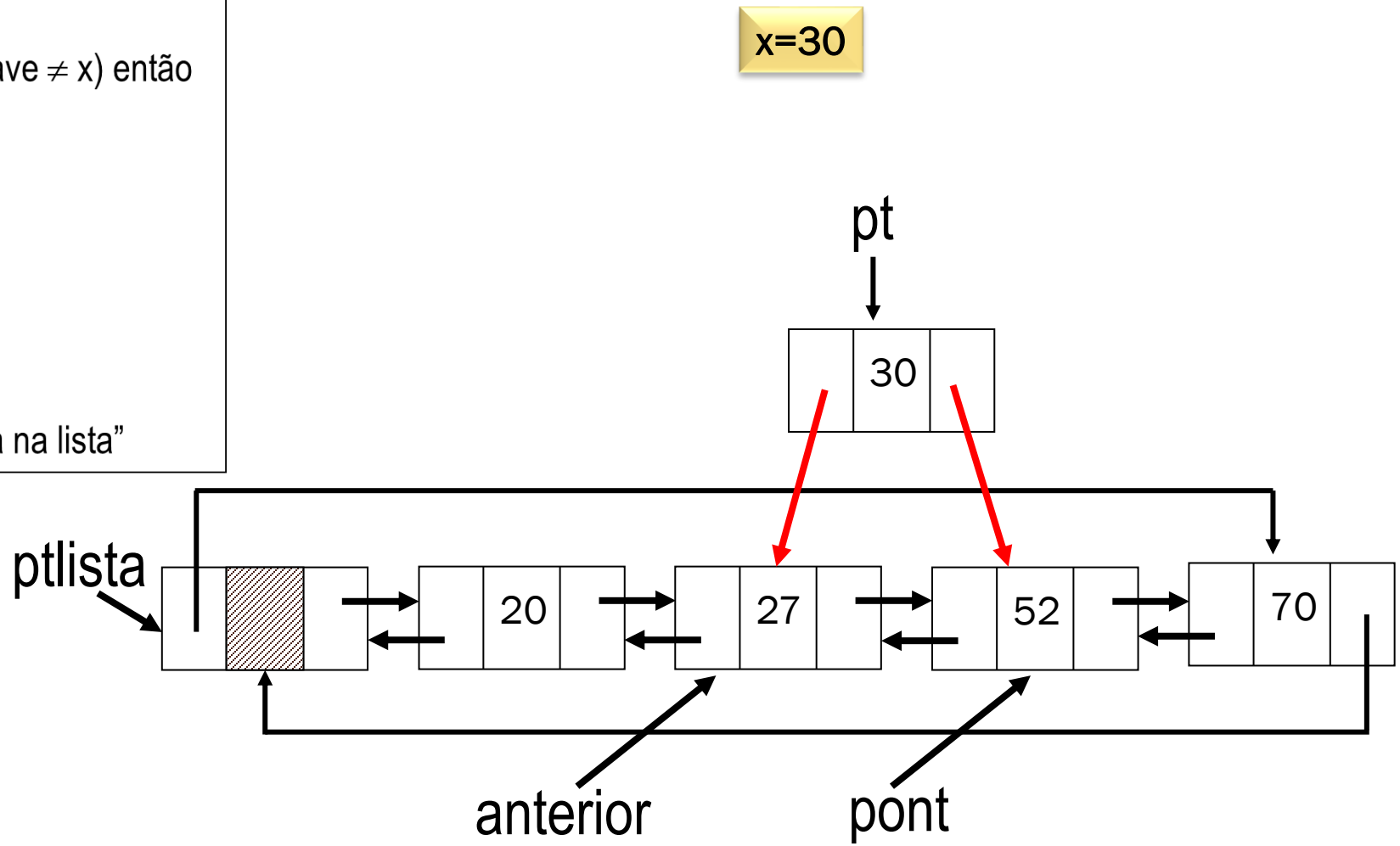
```
Insere(x)
    pont ← Busca(x)
    se (pont = ptlista) ou (pont↑.chave ≠ x) então
        anterior ← pont↑.ant
        ocupar(pt)
        pt↑.chave ← x
        pt↑.ant ← anterior
        pt↑.post ← pont
        anterior↑.post ← pt
        pont↑.ant ← pt
    senão "Elemento já se encontra na lista"
```



Lista Linear Duplamente Encadeada

Inserção

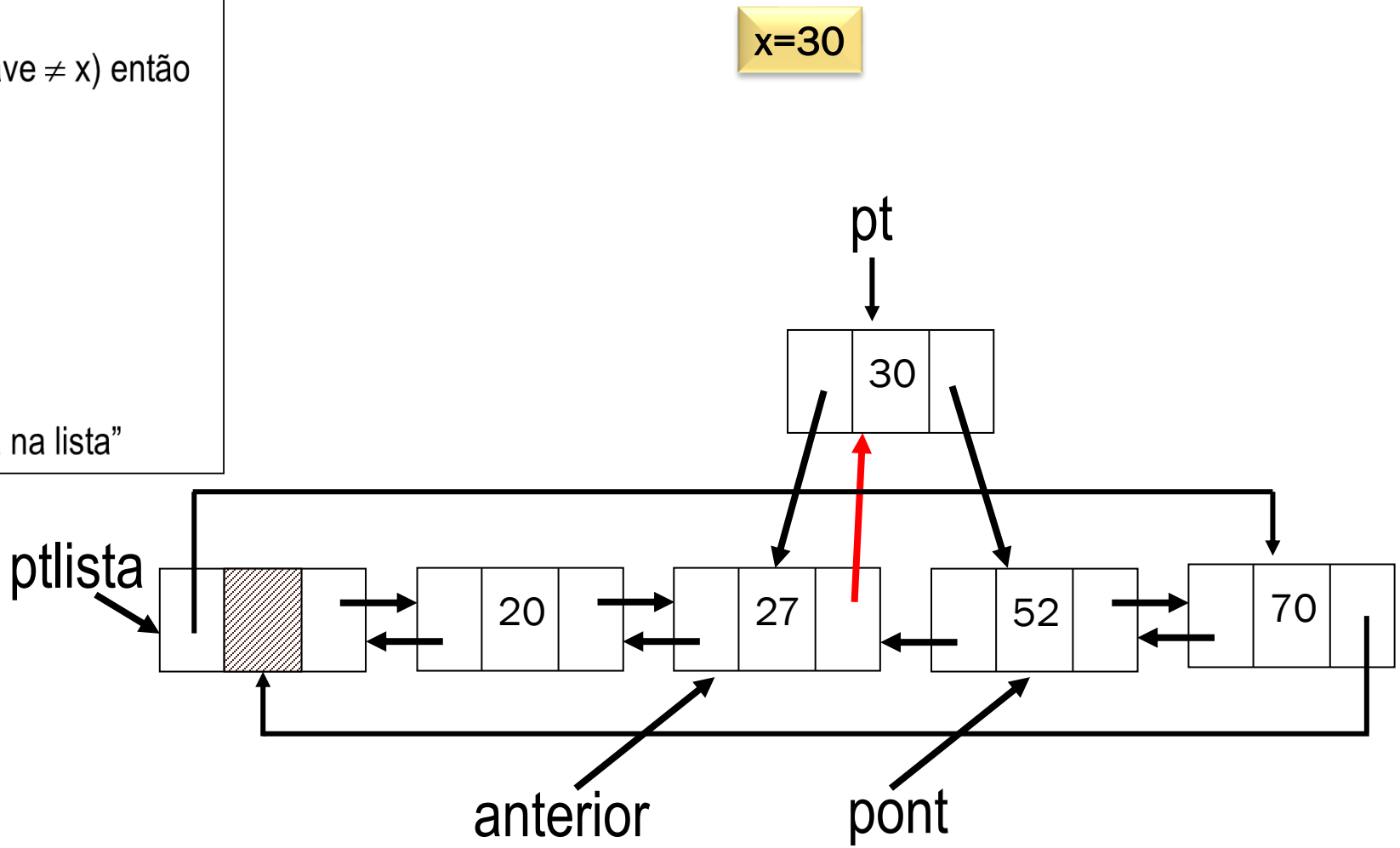
```
Insere(x)
    pont ← Busca(x)
    se (pont = ptlista) ou (pont↑.chave ≠ x) então
        anterior ← pont↑.ant
        ocupar(pt)
        pt↑.chave ← x
        pt↑.ant ← anterior
        pt↑.post ← pont
        anterior↑.post ← pt
        pont↑.ant ← pt
    senão "Elemento já se encontra na lista"
```



Lista Linear Duplamente Encadeada

Inserção

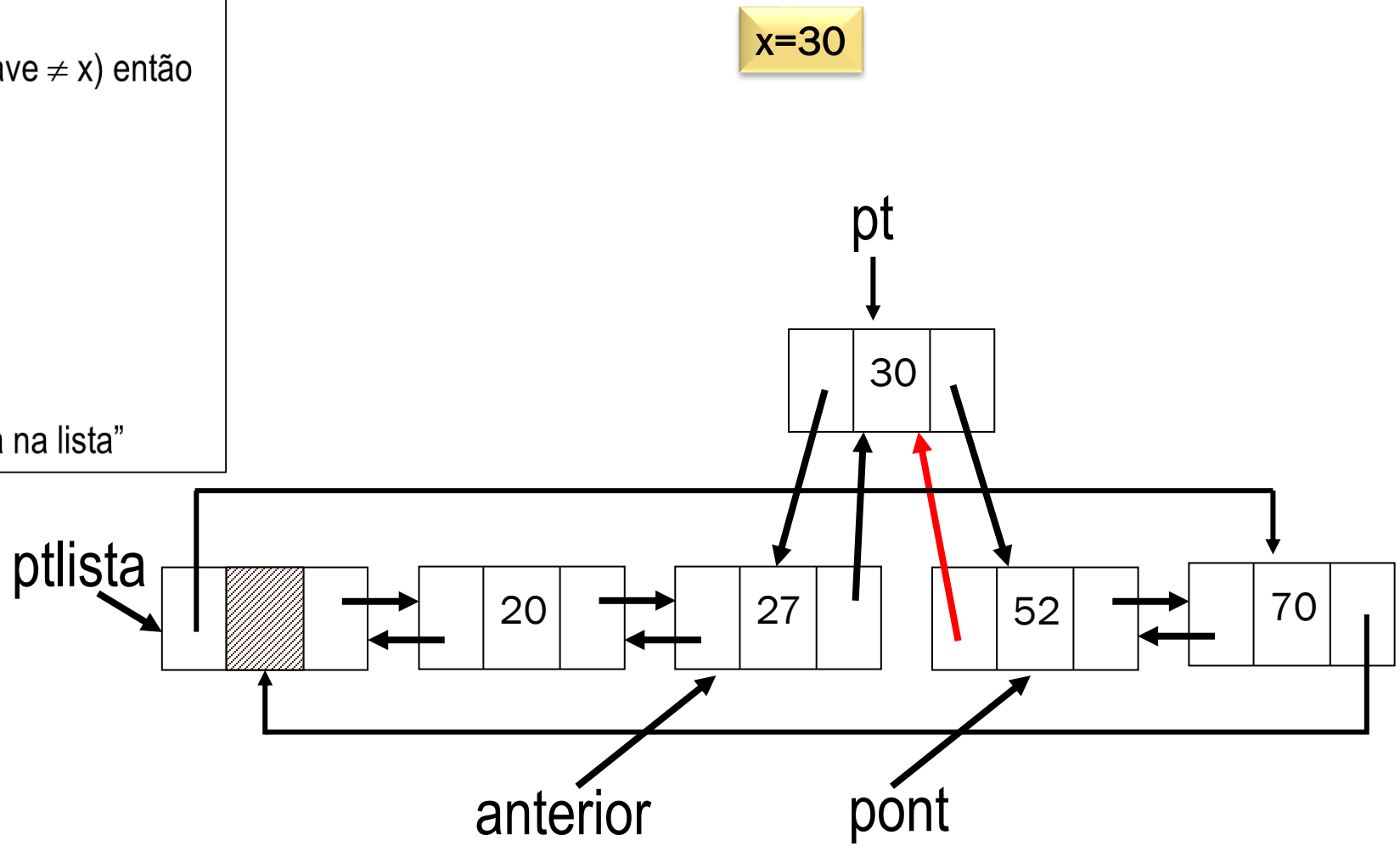
```
Insere(x)
    pont ← Busca(x)
    se (pont = ptlista) ou (pont↑.chave ≠ x) então
        anterior ← pont↑.ant
        ocupar(pt)
        pt↑.chave ← x
        pt↑.ant ← anterior
        pt↑.post ← pont
        anterior↑.post ← pt
        pont↑.ant ← pt
    senão "Elemento já se encontra na lista"
```



Lista Linear Duplamente Encadeada

Inserção

```
Insere(x)
    pont ← Busca(x)
    se (pont = ptlista) ou (pont↑.chave ≠ x) então
        anterior ← pont↑.ant
        ocupar(pt)
        pt↑.chave ← x
        pt↑.ant ← anterior
        pt↑.post ← pont
        anterior↑.post ← pt
        pont↑.ant ← pt
    senão "Elemento já se encontra na lista"
```

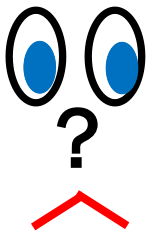


Lista Linear Duplamente Encadeada

Inserção

```
Insere(x)
    pont ← Busca(x)
    se (pont = ptlista) ou (pont↑.chave ≠ x) então
        anterior ← pont↑.ant
        ocupar(pt)
        pt↑.chave ← x
        pt↑.ant ← anterior
        pt↑.post ← pont
        anterior↑.post ← pt
        pont↑.ant ← pt
    senão “Elemento já se encontra na lista”
```

Complexidade ?



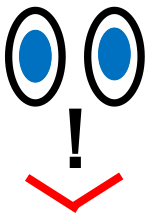
Lista Linear Duplamente Encadeada

Inserção

```
Insere(x)
    pont ← Busca(x)
    se (pont = ptlista) ou (pont↑.chave ≠ x) então
        anterior ← pont↑.ant
        ocupar(pt)
        pt↑.chave ← x
        pt↑.ant ← anterior
        pt↑.post ← pont
        anterior↑.post ← pt
        pont↑.ant ← pt
    senão “Elemento já se encontra na lista”
```

Busca + Comandos em θ(1)

Mesma complexidade da busca



Lista Linear Duplamente Encadeada

